Guía práctica · Subir nuestro proyecto a un repositorio propio de GitHub

MF0491_3 · Programación web en el entorno cliente

Durante el curso hemos clonado un repositorio que contenía el HTML y CSS de nuestra aplicación. Después, cada uno ha desarrollado su propio código JavaScript.

Ahora vamos a subir nuestro trabajo a un repositorio de nuestra propia cuenta de GitHub para, posteriormente, publicarlo en Vercel.

Paso 1. Comprobar a qué repositorio estamos conectados

Abre tu proyecto en Visual Studio Code y accede a la terminal desde Terminal → New Terminal.

Asegúrate de estar dentro de la carpeta de tu proyecto.

Ejecuta:

```
None
git remote -v
```

Este comando muestra los repositorios remotos asociados a nuestro proyecto.

Por ejemplo, podrías obtener:

```
None
origin  https://github.com/profesora/ejercicio-js.git (fetch)
origin  https://github.com/profesora/ejercicio-js.git (push)
```

¿Qué significa?

- `origin`: es el nombre que Git utiliza habitualmente para identificar el repositorio remoto.
- `fetch`: dirección desde la que podemos obtener cambios.
- `push`: dirección a la que podemos enviar nuestros cambios.

Observa: actualmente nuestro proyecto está conectado al repositorio que clonamos para comenzar el ejercicio.

Paso 2. ¿Tenemos que ejecutar `git init`?

No.

Cuando clonamos un repositorio mediante `git clone`, Git ya prepara la carpeta como un repositorio Git y configura normalmente el remoto `origin`.

Por tanto, no necesitamos volver a ejecutar:

```
None
git init
```

Tampoco debemos utilizar `git remote add origin`, porque ya tenemos un remoto con ese nombre.

Lo que vamos a hacer es cambiar la dirección del repositorio remoto para que apunte a nuestra propia cuenta de GitHub.

Paso 3. Crear nuestro repositorio en GitHub

1. Accede a Crear un repositorio en GitHub.
2. Escribe un nombre, por ejemplo, `mi-agenda-tareas`.
3. Selecciona Public para que podamos consultar el proyecto.
4. No marques las opciones para añadir README, `.gitignore` o licencia: queremos empezar con un repositorio vacío.
5. Pulsa Create repository.

GitHub mostrará una dirección similar a:

```
None
https://github.com/TU-USUARIO/mi-agenda-tareas.git
```

Copia la dirección de tu repositorio. La necesitaremos en el siguiente paso.

Paso 4. Cambiar el repositorio remoto

Volvemos a la terminal de Visual Studio Code.

Ejecuta este comando, sustituyendo la dirección por la de tu nuevo repositorio:

```
None
git remote set-url origin
https://github.com/TU-USUARIO/mi-agenda-tareas.git
```

¿Qué hace este comando?

- `git remote`: permite gestionar las conexiones con repositorios remotos.
- `set-url`: cambia la dirección de una conexión existente.
- `origin`: es la conexión que queremos modificar.
- La URL: es la dirección de nuestro nuevo repositorio.

No se eliminan nuestros archivos ni los cambios que hemos realizado en JavaScript. Únicamente cambiamos el destino de los próximos envíos.

Documentación de GitHub

Comprobamos que se ha cambiado correctamente

Vuelve a ejecutar:

```
None
git remote -v
```

Ahora debe aparecer una dirección similar a esta:

```
None
origin  https://github.com/TU-USUARIO/mi-agenda-tareas.git
(fetch)
origin  https://github.com/TU-USUARIO/mi-agenda-tareas.git (push)
```

Si aparece tu usuario de GitHub, ya tienes conectado el proyecto a tu repositorio.

Paso 5. Cambiar el nombre de la rama a `main`

Primero podemos comprobar cómo se llama nuestra rama actual:

```
None
git branch --show-current
```

Si aparece `master`, vamos a cambiar su nombre por `main`:

```
None
git branch -M main
```

¿Qué hace? Cambia el nombre de la rama actual a `main`, que es el nombre que utilizaremos para subir nuestro proyecto.

Este comando no elimina los archivos ni los cambios que hemos realizado.

Si ya estamos en `main`, no necesitamos cambiar nada.

Paso 6. Preparar los archivos para subirlos

Ahora vamos a guardar los cambios realizados en nuestro repositorio local.

6.1. Comprobar los archivos modificados

```
None
git status
```

Git nos indicará qué archivos hemos modificado desde el último commit.

6.2. Añadir los archivos

```
None
git add .
```

El punto `.` indica que queremos preparar todos los cambios de la carpeta actual y sus subcarpetas.

6.3. Crear un commit

```
None
git commit -m "Completar ejercicio con JavaScript"
```

¿Qué es un commit? Es un registro de los cambios que hemos realizado en nuestro proyecto.

Si Git indica `nothing to commit, working tree clean`, significa que no tenemos cambios pendientes de registrar. Podemos continuar con el siguiente paso.

Paso 7. Subir el proyecto a GitHub

Ejecutamos:

```
None
git push -u origin main
```

Este comando envía los commits de nuestra rama `main` al repositorio remoto que acabamos de configurar.

La opción `-u` establece la relación entre nuestra rama local y la remota, lo que facilitará los próximos envíos.

Documentación de GitHub

GitHub puede solicitar que iniciemos sesión o autoricemos la operación.

Comprobación

Abre tu repositorio en GitHub y actualiza la página.

Deberías encontrar archivos similares a estos:

```
None
mi-agenda-tareas/
├── index.html
├── styles.css
├── app.js
└── images/
```

Comprueba especialmente que `app.js` contiene el código que has desarrollado.

Resumen de comandos

Esta es la secuencia completa para un proyecto que ya hemos clonado y que tiene configurado `origin`.

Comandos para la terminal

Copiar

```
None
git remote -v

git remote set-url origin
https://github.com/TU-USUARIO/mi-agenda-tareas.git
```

```
git remote -v

git branch --show-current

git branch -M main

git status

git add .

git commit -m "Completar ejercicio con JavaScript"

git push -u origin main
```

Importante: cambia `TU-USUARIO` y `mi-agenda-tareas` por los datos de tu repositorio. Ejecuta las instrucciones en orden y comprueba el resultado de cada una antes de continuar.

Paso 8. Preparados para publicar en Vercel

Una vez que los archivos estén en nuestro repositorio de GitHub, podremos importar el proyecto en Vercel.

Como hemos trabajado con HTML, CSS y JavaScript, no necesitaremos instalar dependencias ni configurar un proceso de compilación.

El objetivo de la siguiente actividad será conseguir una dirección pública desde la que cualquier persona pueda utilizar nuestra aplicación.

Comprobación final

Lista de comprobación

0 de 8

He comprobado mi repositorio remoto con git remote -v.
He creado mi propio repositorio vacío en GitHub.
He cambiado origin para que apunte a mi repositorio.
He comprobado que mi rama se llama main.
He preparado los cambios con git add .
He creado un commit de mi trabajo.
He subido los archivos con git push -u origin main.
He comprobado que mi JavaScript aparece correctamente en GitHub.

Recuerda: clonar es descargar una copia de un repositorio; crear un commit es registrar nuestros cambios; hacer `push` es enviarlos a GitHub. Son tres operaciones diferentes y todas forman parte del trabajo habitual con Git.